

Intelligent System Development

Lecture 02 — Data Representation and Intelligent Systems

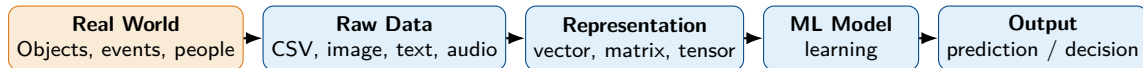
Dinh Que Tran, Ph.D., Assoc. Prof.

Department of Information Technology
Posts and Telecommunications Institute of Technology (PTIT)

quetd@ptit.edu.vn, tdque@yahoo.com

2026

From Real-World Data to Machine-Learning Representations



Central Question

How do we transform heterogeneous real-world data into representations that an intelligent system can process, learn from, evaluate, and deploy?

Why Data Representation Matters

The same information can have different representations

- Raw image
- Pixel matrix
- Normalized tensor
- CNN feature tensor
- Feature vector

A model expects a specific representation

- Shape
- Data type
- Value range
- Ordering
- Semantic meaning
- Preprocessing

Key Principle

A file format is not the same thing as a machine-learning representation.

File $\neq$ Representation $\neq$ Model Input

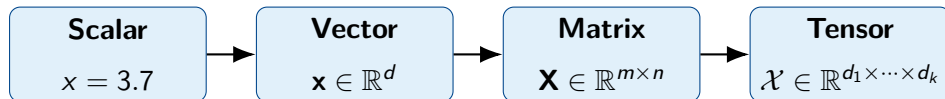
Different Forms of Data

Data form	Typical files	Example
Tabular	CSV, Excel, SQL	customer records
Image	JPEG, PNG, TIFF	medical image
Text	TXT, JSON, PDF	document / review
Audio	WAV, MP3	speech recording
Video	MP4, AVI	surveillance video
Time series	CSV, database	sensor measurements
Graph	JSON, edge list	social network

Common Goal

Regardless of the original data form, an intelligent system eventually needs a numerical representation.

From Scalar to Vector, Matrix, and Tensor



- **Scalar**: one numerical value.
- **Vector**: an ordered collection of values.
- **Matrix**: a two-dimensional array.
- **Tensor**: a general multi-dimensional array.

Important

Vectors, matrices, and tensors are not different kinds of “intelligence” — they are different mathematical structures used to organize numerical information.

Vector Representation

A vector represents one ordered set of numerical values:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^d$$

Example: one customer

$$\mathbf{x} = \begin{bmatrix} 25 \\ 4.2 \\ 12 \\ 0.73 \end{bmatrix}$$

where the components might represent:

Age Spending Purchases Loyalty

Matrix Representation

Suppose we have N samples and d features.

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{Nd} \end{bmatrix} \in \mathbb{R}^{N \times d}$$

N = number of samples

$\mathbf{x}_i$ = one sample

d = number of features

$\mathbf{x}_{:j}$ = one feature

Typical tabular representation

$$\mathbf{X} \in \mathbb{R}^{N \times d}$$

Tensor Representation

A tensor generalizes vectors and matrices to higher dimensions.

$$\mathcal{X} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_k}$$

Data	Typical shape	Meaning
Tabular batch	$B \times d$	samples $\times$ features
Image	$C \times H \times W$	channels $\times$ height $\times$ width
Image batch	$B \times C \times H \times W$	batch of images
Text IDs	$B \times T$	batch $\times$ tokens
Text embeddings	$B \times T \times d$	batch $\times$ tokens $\times$ embedding
Video	$B \times C \times T \times H \times W$	video batch

Core idea

The meaning of every axis must be understood, not merely its size.

CSV: From File to Feature Matrix

CSV File

Age	BMI	BP	Glu	Class
21	22.1	120	95	0
35	27.4	130	110	1
48	31.2	145	160	1
62	29.8	150	180	2

4 rows
4 samples

4 feature columns
inputs

+

Class
target

ML Representation

$$\mathbf{X} = \begin{bmatrix} 21 & 22.1 & 120 & 95 \\ 35 & 27.4 & 130 & 110 \\ 48 & 31.2 & 145 & 160 \\ 62 & 29.8 & 150 & 180 \end{bmatrix}$$

$$\mathbf{X} \in \mathbb{R}^{4 \times 4}$$

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 2 \end{bmatrix}$$

$$\mathbf{y} \in \mathbb{R}^4$$

CSV: Batch Representation

For tabular learning:

$$\mathbf{X} \in \mathbb{R}^{N \times d}$$

where: N = total number of samples, d = number of input features.
During mini-batch training: $\mathbf{X}_{batch} \in \mathbb{R}^{B \times d}$ where B is the batch size.

Example

$$N = 10\,000, \quad d = 20, \quad B = 32$$

Full dataset:

$$\mathbf{X} \in \mathbb{R}^{10000 \times 20}$$

One training batch:

$$\mathbf{X}_{batch} \in \mathbb{R}^{32 \times 20}$$

Categorical Data Representation

Not every CSV value is naturally numerical.

Example:

Raw value	Problem	Possible representation
Male / Female	categorical	one-hot / embedding
Red / Green / Blue	categorical	one-hot / embedding
Low / Medium / High	ordinal	ordered encoding
Hanoi / Hue / Da Nang	nominal	one-hot / embedding

Common mistake

Encoding categories as 1, 2, 3 does not automatically mean that category 3 is “greater” than category 1.

Image Representation: Pixels

An image is a structured collection of pixels.

Grayscale image: $X \in \mathbb{R}^{H \times W}$

RGB image: $X \in \mathbb{R}^{H \times W \times 3}$

or, in channel-first form:

$$X \in \mathbb{R}^{3 \times H \times W}$$

Pixel values

For an 8-bit RGB image: $x_{hwc} \in \{0, 1, \dots, 255\}$

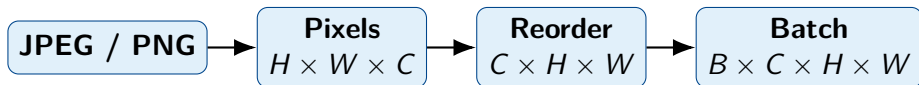
After conversion to floating point: $x_{hwc} \in [0, 1]$

or after a different normalization: $x_{hwc} \in [-1, 1]$.

Image Representation: Batch Tensor

A single RGB image: $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$

A mini-batch: $\mathcal{X} \in \mathbb{R}^{B \times C \times H \times W}$



PyTorch example

$$B = 32, \quad C = 3, \quad H = W = 224$$

```
torch.Size([32,3,224,224])
```

Image Data: Channel Ordering

Channel-last

$$H \times W \times C$$

Example:

$$224 \times 224 \times 3$$

Common in:

- NumPy image arrays
- OpenCV
- TensorFlow/Keras

Channel-first

$$C \times H \times W$$

Example:

$$3 \times 224 \times 224$$

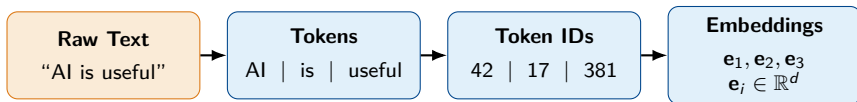
Common in:

- PyTorch
- Many CNN implementations

Practical lesson

Always check the expected tensor convention before passing data to a model.

Text Representation: From Symbols to Numbers



Key Idea

Text is symbolic. Machine-learning models operate on numerical representations, so text must be transformed into **tokens**, **integer IDs**, and **ultimately vectors**.

Text Representation: Token IDs

1. Vocabulary

A tokenizer maps each token to an integer:

$$f : \mathcal{V} \rightarrow \{0, 1, \dots, V - 1\}$$

Example:

AI $\rightarrow$ 42

is $\rightarrow$ 17

useful $\rightarrow$ 381

2. Sequence and Batch

For one sentence:

$$\mathbf{x}_{id} = [x_1, \dots, x_T] \in \mathbb{N}^T$$

For a batch:

$$\mathbf{X}_{id} \in \mathbb{N}^{B \times T}$$

Example:

$$B = 32, \quad T = 128$$

$$\mathbf{X}_{id} \in \mathbb{N}^{32 \times 128}$$

Text Representation: Token IDs (cont)

What do the numbers mean?

Token IDs are **integer indices**, not semantic values.

“AI is useful” $\rightarrow$ [12, 45, 81]

$12 = \text{ID}(\text{AI}), \quad 45 = \text{ID}(\text{is}), \quad 81 = \text{ID}(\text{useful})$

ID $\rightarrow$ Embedding Lookup $\rightarrow$ Dense Vector

Text Representation: Embeddings

$$x_i \xrightarrow{\text{Embedding Layer}} \mathbf{e}_i \in \mathbb{R}^d$$

For a sequence of T tokens:

$$\mathbf{E} = \begin{bmatrix} \mathbf{e}_1^T \\ \mathbf{e}_2^T \\ \vdots \\ \mathbf{e}_T^T \end{bmatrix} \in \mathbb{R}^{T \times d}$$

For a batch:

$$\mathcal{E} \in \mathbb{R}^{B \times T \times d}$$

Sentences have different lengths

“AI is useful” 3 tokens

“AI is useful for healthcare” 5 tokens

To create a batch, shorter sequences are padded

$$\mathbf{X}_{id} = \begin{bmatrix} 12 & 45 & 81 & 0 & 0 \\ 12 & 45 & 81 & 73 & 19 \end{bmatrix}$$

$$\underbrace{0}_{\text{padding}} \quad B = 2, \quad T = 5$$

Attention mask

From Data to Numerical Tensors

Data Modality	Numerical Representation	Typical Batch Shape
Tabular	Feature matrix	$B \times d$
Image	Pixel tensor	$B \times C \times H \times W$
Text	Token IDs	$B \times T$
Text	Token embeddings	$B \times T \times d$
Audio	Sample sequence	$B \times T$
Video	Spatiotemporal tensor	$B \times C \times T \times H \times W$
Time series	Feature sequence	$B \times T \times d$

Key Idea

Data Representation Is More Than Shape

For a tensor $\mathcal{X}$, we should understand:

- 1 **Shape** — What are the dimensions?
- 2 **Semantics** — What does each axis mean?
- 3 **Dtype** — How are values stored?
- 4 **Range** — What values are valid?
- 5 **Scale** — Are numerical magnitudes comparable?
- 6 **Ordering** — Does position have meaning?
- 7 **Missing values** — How are they represented?
- 8 **Encoding** — How were categories/tokens converted?

Example

$$\mathcal{X} \in \mathbb{R}^{32 \times 224 \times 224 \times 3}$$

The shape alone does not tell us whether the tensor is RGB or BGR, normalized or unnormalized, channel-first or channel-last.

Data Types and Value Ranges

Dtype	Typical use	Example
uint8	raw image pixels	$0, \dots, 255$
int64	token/class IDs	$0, 1, 2, \dots$
float32	neural-network input	real values
float64	scientific computation	high precision
bool	masks	True / False

Image example $X_{\text{raw}} \in \{0, \dots, 255\}^{H \times W \times 3}$

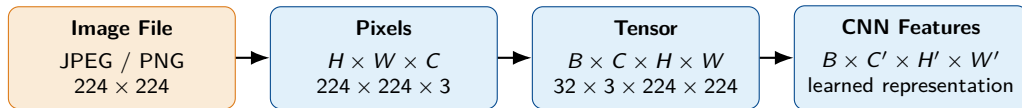
After normalization: $X_{\text{norm}} \in [0, 1]^{3 \times H \times W}$

or: $X_{\text{norm}} \in [-1, 1]^{3 \times H \times W}$

Lesson

Preprocessing changes the numerical representation while preserving the underlying information as much as possible.

Example: Representing an Image



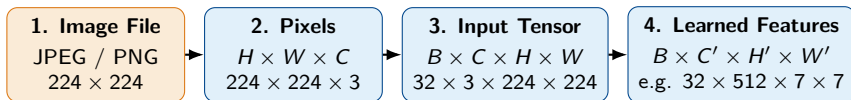
What does each dimension mean?

$B = \text{batch}, \quad C = \text{channels}, \quad H = \text{height}, \quad W = \text{width}$

Key Idea

An image is not “a picture” to the neural network. It is a **structured tensor of numerical values**.

Example: How an Image Becomes Model Input



A Pixel is a Vector

For an RGB image:

$$I_{i,j} = [R_{i,j}, G_{i,j}, B_{i,j}]$$

Example:

$$[120, 85, 200] \rightarrow [0.47, 0.33, 0.78]$$

Tensor Dimensions

B = batch, C = channels

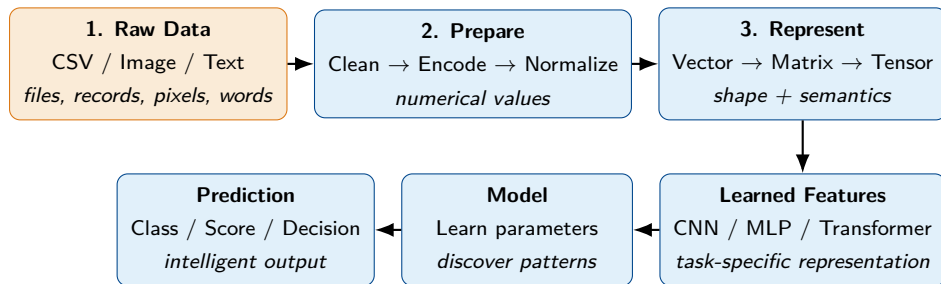
H = height, W = width

PyTorch:

$$B \times C \times H \times W$$

Key Idea

Representation Transformations



Key Idea

Representation is not a single preprocessing operation. It evolves from raw data to model inputs and finally to learned features.

Common Representation Errors

- **Wrong shape** $(32, 224, 224, 3)$ vs. $(32, 3, 224, 224)$
- **Wrong dtype**: Integer data passed where floating-point input is expected.
- **Wrong value range** $[0, 255]$ vs. $[0, 1]$ vs. $[-1, 1]$
- **Wrong categorical encoding**
Treating categories as meaningful numerical quantities.
- **Missing values**
NaN, empty strings, zeros, or sentinel values interpreted incorrectly.
- **Data leakage**
Information from validation/test data enters preprocessing or training.
- **Inconsistent preprocessing**
Training and inference use different transformations.

Representation Contract

Every ML project should explicitly document its input representation.

Property	Question
Data source	Where did the data come from?
File format	CSV, JPEG, JSON, TXT, ...?
Samples	How many observations?
Features	What does each feature mean?
Shape	What is the tensor shape?
Dtype	How are values stored?
Range	What values are valid?
Encoding	How are categorical/text values encoded?
Normalization	How are numerical values scaled?
Missing values	How are missing observations handled?
Split	How are train/validation/test sets created?
Model input	What exact tensor does the model receive?

Example: Three Data Modalities

Tabular Data

CSV / Excel

$$N \times d$$

Rows = samples



$$B \times d$$

Model input



MLP

Image

JPEG / PNG

$$H \times W \times 3$$

Pixels



$$B \times 3 \times H \times W$$

Model input



CNN

Text

TXT / JSON

$$B \times T$$

Token IDs



$$B \times T \times d$$

Embeddings



Transformer

Example: Three Data Modalities (cont)

Key Idea

Different modalities have different structures, but intelligent systems convert them into **numerical representations with explicit shapes**.

Representation in Python

```
# Tabular data
X.shape
# (1000, 20)

# Image batch
images.shape
# (32, 3, 224, 224)

# Text token IDs
input_ids.shape
# (32, 128)

# Text embeddings
embeddings.shape
# (32, 128, 768)
```

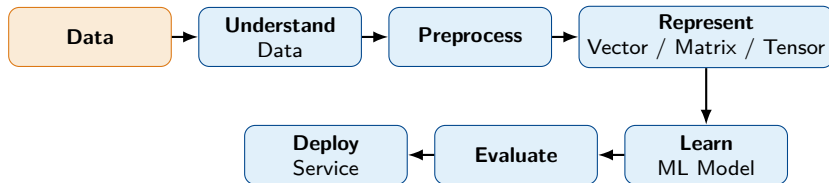
Developer Habit - Before training, inspect

- shape
- dtype
- min/max
- mean/std

Key Idea

The **shape** tells you how the data is organized; the **dtype** and numerical range tell you how it is stored.

From Data Representation to Intelligent System

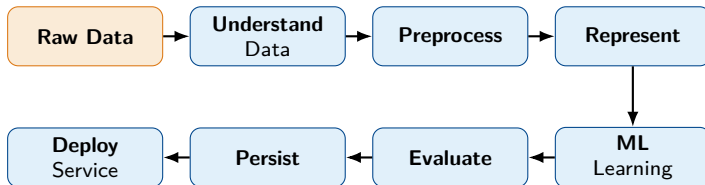


Engineering Principle

A successful intelligent system requires both a good model and a correct data representation pipeline.

Assignment 02: From Raw Data to a Deployable ML System

“A model is only as reliable as the data and evaluation process behind it.”



Main Objective

Develop a reproducible machine-learning pipeline and turn it into a simple deployable intelligent service.

Assignment 02: Required Deliverables

Each student/team must provide:

① **Dataset description**

Source, size, attributes, data types, and target variable.

② **Data representation**

Explain the vector/matrix/tensor representation used by the model.

③ **Preprocessing pipeline**

Cleaning, encoding, normalization, resizing, tokenization, etc.

④ **Train/validation/test strategy**

Explain the split and prevent data leakage.

⑤ **ML model and Evaluation**

Train at least one appropriate baseline model.

Report appropriate metrics and analyze the results.

⑥ **Persistence and Deployment**

Save the trained model and preprocessing configuration.

Provide a simple API, web interface, or application endpoint.

Assignment 02: Representation Checklist

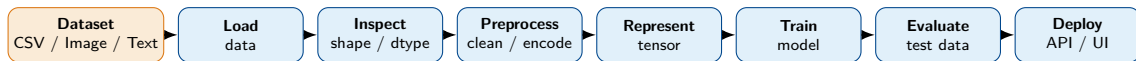
Before training, students should be able to answer:

- 1 What is the original data format?
- 2 What does one sample look like?
- 3 What is the input feature representation?
- 4 What are the dimensions of the input?
- 5 What does each dimension mean?
- 6 What is the data type?
- 7 What is the numerical range?
- 8 How are categorical or textual values encoded?
- 9 How are missing values handled?
- 10 What preprocessing is applied?
- 11 Is the same preprocessing used during inference?
- 12 What exact tensor enters the model?

Minimum requirement

Students must be able to print and explain the shape and dtype of the data entering their

Assignment 02: Example End-to-End Pipeline



Final Goal

The assignment is not simply “train a model.”

It is to build a small, reproducible, and deployable **intelligent system**.

Key Takeaways

- ➊ Data comes in many forms: tabular, image, text, audio, video, time series, and graphs.
- ➋ A file format is a storage mechanism, not a learning representation.
- ➌ Numerical data can be organized as scalars, vectors, matrices, and tensors.
- ➍ Different modalities produce different tensor shapes.
- ➎ Shape alone is insufficient: semantics, dtype, range, ordering, and preprocessing also matter.
- ➏ Correct representation is essential for correct model training and inference.
- ➐ Data representation must be reproducible from training through deployment.

Good Data + Good Representation + Good Evaluation → Reliable Intelligent System

From Data Representation to Model Development

Data → Representation → Model → Evaluation → Deployment

Assignment 02

Build the complete pipeline from raw data to a deployable intelligent service.